TermNav Design and Evaluation of a Terminal File Manager

1st [Author 1]
[Department]
[Institution]
[City and Country]
[Email Address]

2nd [Author 2]
[Department]
[Institution]
[City and Country]
[Email Address]

3rd [Author 3]
[Department]
[Institution]
[City and Country]
[Email Address]

4th [Author 4]
[Department]
[Institution]
[City and Country]
[Email Address]

5th [Author 5]
[Department]
[Institution]
[City and Country]
[Email Address]

Abstract—TermNav is a keyboard driven terminal file manager implemented in C11 with ncursesw and POSIX threads. It addresses fragmented file workflows through a common interface for navigation, recursive filename search, read only previews, recoverable deletion, operation monitoring and storage inspection. This paper presents a design and implementation case study of the project, relates each problem to its implementation mechanism, and evaluates functional behavior using repository checks. The design assigns terminal rendering to one thread, moves selected long running tasks to workers, bounds resource consumption, and uses filesystem operations that reject unintended replacement. A Home view exposes common folders and drive capacity, while a Windows launcher translates user folder locations into WSL paths. Verification at the inspected revision passed five C test executables and six Python integration suites; the filesystem, preview and worker executable reported 4,956 checks. These results support correctness for the exercised scenarios. They do not establish productivity gains, comparative speed or universal portability. The study identifies remaining limitations in transactional recovery, trash interoperability, terminal graphics compatibility and evaluation coverage.

Index Terms— terminal file management; ncurses; filesystem safety; asynchronous operations; recursive search; WSL; terminal previews

I. Introduction

Managing files from a terminal often requires separate commands to list directories, search for names, inspect contents, copy data and estimate storage consumption. Each command is useful independently, but the user must repeatedly reconstruct context: the current directory, the selected item, the destination and the state of a background operation. Destructive commands also create a recovery problem when a mistaken deletion removes data immediately. TermNav brings these activities into a persistent visual workspace [1].

The project combines a parent directory pane, an active file list and a preview pane with a Home dashboard. Its purpose is practical integration rather than a new search algorithm or a claim to outperform established tools. The paper asks how a terminal interface can preserve navigation context, expose long running work, reduce specific risks of file operations and adapt previews to terminal capabilities. The contribution is a traceable account of the implemented mechanisms and the functional evidence available for them.

II. Related Work and Problem Definition

Yazi establishes that modern terminal file managers can combine asynchronous tasks, previews, terminal image protocols and extensive keyboard workflows. Its official implementation uses Rust and includes plugins and filesystem integrations [2]. TermNav occupies the same broad application category. Its distinguishing focus in this study is the combination of a C implementation, explicit resource bounds, a Home view and Windows user folder integration through WSL. Feature overlap does not establish novelty, and no direct performance comparison is reported.

The interface uses ncursesw to draw terminal windows and handle wide characters. The ncurses threading documentation describes limited threading support rather than automatic thread safety [3]. TermNav therefore centralizes curses calls in the interface thread. WSL provides a practical route to Windows files through Linux paths, including mappings such as /mnt/c [4]. This supports Windows users without making TermNav a native Windows application.

The requirements are expressed below as observable engineering goals. Expected workflow benefits are design intentions; the evaluation does not measure human time savings.

TABLE I PROBLEMS AND CORRESPONDING IMPLEMENTATION MECHANISMS

Problem	Implemented response	Observable behavior
Repeated loss of location and selection	Three pane browser, Home shortcuts and session history	The user can return to prior locations and inspect an item in context.
Unknown file location	Local fuzzy filtering and recursive filename search	Names can be narrowed locally or discovered beneath a search root.
Accidental deletion or replacement	Private trash, batch restore and no replacement operations	Deleted items can be restored; conflicting destinations return errors.
Opaque long running operations	Worker queue and operations dashboard	Progress and state are visible; supported jobs can pause or cancel.
Unclear storage consumption	Drive capacity bars and recursive disk usage explorer	Volume availability and directory allocation are shown separately.

Problem	Implemented response	Observable behavior
Repeated switching to viewer tools	Text, binary, image, PDF and archive previews	Supported content can be inspected without leaving the file manager.

A. Scope and Assumptions

The analyzed artifact is repository commit 13599afee55c65c6a8e44523d6c1e87c3806f8f4, inspected on 6 October 2026 [1]. The supported execution target is Linux, including Linux inside WSL. Image and PDF preview capabilities depend on optional helpers and terminal support. The application operates with the user's filesystem permissions; it neither grants access to unreadable directories nor supplies a remote filesystem abstraction.

III. Architecture and State Management

TermNav separates interface state, filesystem operations and content conversion. The main application state records the current directory, visible entries, selection, marked items, active panel and worker status. The input layer translates keys into actions, and the rendering layer draws the active view. The central control loop consumes input, observes worker updates and redraws the terminal. Only this thread calls curses, avoiding simultaneous terminal writes from background tasks [1, 3].

A. Major Code Blocks

src/main.c coordinates initialization, the event loop and cleanup. src/input.c implements navigation, prompts and action dispatch. src/ui.c renders the browser, Home view and modal panels. src/fs_ops.c supplies directory loading and low level file operations. These shared functions keep navigation and filesystem behavior consistent across the Home view, browser and search results.

src/async_ops.c manages queued file operations, while src/trash.c supplies deletion records and restoration. src/search.c performs recursive discovery and src/usage.c measures directory allocation. src/history.c restores session navigation state, and src/commands.c exposes existing actions through the command palette. Drive discovery and capacity reporting live in src/drives.c. This organization provides identifiable implementation boundaries without requiring an independent service or database.

src/preview.c handles bounded text and binary inspection. src/rich_preview.c coordinates helper execution, src/image_terminal.c negotiates terminal graphics capabilities, and src/sixel.c encodes native image output. scripts/preview_helper.py converts supported content into bounded preview data. termnav.ps1 resolves Windows user folder locations and passes translated paths to the Linux process. The helper boundary is deliberate: optional content libraries remain outside the main C executable.

B. Concurrency and Resource Bounds

File mutations use one active worker and a queue of up to 16 pending jobs. Completed operation history retains 32 session records. Search, disk usage and rich preview have separate worker paths. Worker owned path snapshots avoid dependence on a directory that the user subsequently changes. Atomic flags communicate cancellation and simple progress, while mutexes protect shared collections. Shutdown joins workers before their resources are released.

Other bounds include 64 navigation history entries, 1,000 search results, a traversal depth of 128 and a 64 KiB text preview budget. These values bound specific collections or outputs; they do not cap every filesystem operation's duration. Directory enumeration and some preview work remain synchronous. A blocked operating system call can delay cooperative cancellation even though the interface architecture supports background execution.

C. Consistent Directory Changes

Directory loading constructs replacement state before committing a navigation change. If a directory cannot be opened, the previous view remains usable rather than becoming an empty or inconsistent screen. Refresh logic preserves selection by name and marked entries using name and inode identity. A separate visible index supports filtering without discarding the underlying directory metadata. Display sanitization and width aware clipping keep filenames from injecting terminal controls or breaking pane boundaries.

IV. Navigation Search and Storage Inspection

A. Home and Directory History

The Home view provides shortcuts to Home, Downloads, Documents and Pictures, alongside a file list, details and compact drive rows. F1 returns to Home, and Tab switches among Home interaction areas. Enter opens the selected item; browser navigation also accepts arrow and familiar letter keys. The folder cards and drive bars serve as entry points into real directories rather than decorative summaries.

The Windows launcher obtains user folder locations from platform settings, including redirected Downloads and OneDrive locations, then converts them with wslpath. This avoids assuming that every user stores Documents under a fixed profile path. The Linux application receives these paths through environment variables. WSL path translation is necessary because a Linux process expects Linux path syntax [4]. Session history keeps directory, selection, filtering and scrolling context. Navigating after going back removes the abandoned forward branch; history is not persisted across restarts.

B. Two Search Scopes

The slash command filters entries in the current directory using subsequence matching. Recursive search, opened with S or the command palette, traverses names beneath the current root in a worker. The matching routine ignores ASCII letter case; it is not complete Unicode case folding. Results can open a directory, preview a file or reveal an item in its containing directory. The user can stop a scan while retaining discoveries, restart it or submit another query.

Traversal uses descriptor based directory access and inspects entries without following directory symbolic links. It skips dotfiles when hidden files are disabled and does not descend into another device. Permission failures and excluded paths therefore affect completeness. The maximum query is 255 bytes, stored results stop at 1,000, and traversal depth is limited to 128. These safeguards prevent unbounded result storage and deep recursion. The result limit does not necessarily end traversal, so a large tree with few matches can still require substantial work.

For N inspected entries and maximum filename length L, a simple upper bound for name matching is O(NL), apart

from filesystem access costs. Stored result memory is bounded by K path records, where K is at most 1,000, plus traversal state. The implementation avoids an indexing database and consequently pays traversal costs for each new scan. It searches filenames rather than file contents. Indexed or full text search would be a separate capability.

C. Capacity and Directory Allocation

Drive discovery combines mount information with statvfs statistics. The displayed occupancy uses total capacity minus space available to an unprivileged user, so reserved blocks contribute to the occupied portion [5]. Duplicate and virtual mounts are filtered, up to 16 drive entries are retained, and the Home view refreshes capacity periodically. A compact layout can show four drive rows when the terminal has sufficient height; a shorter view requires scrolling.

The disk usage explorer performs a separate recursive scan and counts allocated bytes using st_blocks multiplied by 512. Apparent file length and allocated storage are different measurements, especially for sparse files. This view helps identify large directory contents, whereas drive bars describe volume capacity. Permission exclusions, filesystem allocation behavior and traversal limits must be considered when interpreting totals.

V. Recoverable Operations and Rich Previews

A. Collision Protection and Trash Recovery

The operations dashboard exposes copy, move and trash jobs, their progress, queued state and completion errors. Supported workers poll pause and cancellation flags. Navigation can continue while a job runs, while conflicting changes such as rename are protected. This makes operation state inspectable without implying that every system call is interruptible.

Regular file copying writes into an exclusive staging file, checks partial reads and writes, applies supported metadata, synchronizes data and publishes the destination without replacing an existing name. Rename paths use Linux RENAME_NOREPLACE, which rejects an existing destination when the filesystem supports the operation [6]. Descriptor based traversal and refusal to follow directory links reduce specific race and traversal hazards. These are targeted safeguards, not a proof that every hostile filesystem interaction is eliminated.

Trash places payloads in a private location on the source filesystem and stores a registry of original paths, stored paths, timestamps and batch identifiers. Registry ownership and permissions are checked, locking serializes metadata changes, and records are synchronized before the payload rename. Undo restores a trash batch after checking the destination; it can work after restarting the application. This addresses accidental deletion while avoiding silent replacement of a newer file.

The recovery scope is limited. Undo applies to trash, not arbitrary copies, moves or renames. Directory copies may retain entries already completed when cancellation or failure occurs. Copy metadata does not preserve all ownership, ACL or extended attribute information. Metadata and payload changes do not constitute a full power failure transaction. TermNav's private store also does not implement the shared freedesktop Trash specification and does not integrate with Windows Recycle Bin [7].

B. Read Only Content Inspection

Text preview reads a bounded prefix and constructs line positions for scrolling; simple highlighting improves inspection without parsing a complete programming language. Binary preview shows byte offsets, hexadecimal values and printable characters in 16 byte rows. A full screen viewer supports scrolling and panning. These paths do not modify the inspected file.

Rich previews include common raster image formats, extractable text from the first five PDF pages and bounded archive listings. Archives are listed rather than extracted, and PDF inspection does not perform OCR. The parent opens the file read only, validates its identity and passes a descriptor to a helper launched with an argument vector rather than a shell command. Output is validated and capped; timeouts and resource limits constrain helper work. Malformed or unsupported content produces a visible error.

Image rendering negotiates terminal capability. A Sixel capable terminal can display a raster preview up to the helper's 1280 by 960 size bound, with Lanczos resizing and a palette of up to 256 colors. Otherwise the application uses colored character cells. The fallback necessarily has lower spatial resolution because terminal cells contain much less detail than an image raster. Native previews still depend on source resolution, palette reduction, pane size and terminal behavior; the project does not guarantee universally sharp or full resolution images.

VI. Evaluation Method and Results

Evaluation combined source inspection with a fresh run of the repository's make check target on 6 October 2026. The command executed in the project's TermNav Dev WSL environment against the inspected revision. The test harnesses create controlled filesystem cases and exercise terminal behavior through pseudo terminals and protocol checks. This is a functional regression evaluation rather than a user study or benchmark.

All five C test executables and all six Python integration suites completed successfully. The first C executable reported 4,956 filesystem, preview and worker checks. This number is an internal assertion count from that executable; it is not a count of independent participants, workloads or all project tests. Table 2 groups the exercised behavior and the limits of the corresponding evidence. The captured output is retained locally at build/research-paper/verification.log.

TABLE II VERIFICATION EVIDENCE AT THE INSPECTED REVISION

Test group	Exercised behavior	Result and interpretation
Filesystem and application C checks	Filesystem operations, workers, confirmation targets and Home navigation	Pass; controlled cases support implemented behavior.
Feature and search C checks	Trash persistence, collisions, history, usage scans, recursive search bounds and cancellation	Pass; recoverability and boundary cases were exercised.
Sixel C checks	Bounded encoding, palette, raster header and stream termination	Pass; encoding validity under the tested cases.

Test group	Exercised behavior	Result and interpretation
Home and search integration	Folder shortcuts, paths containing spaces, result preview, reveal and hidden files	Pass; interaction paths worked in the test environment.
Terminal and feature integration	Navigation, editing handoff, resizing, signals, terminal restoration and trash across restart	Pass; automated terminal scenarios completed.
Image and helper integration	Native image detail, fallback, detection, cleanup, malformed input and archive truncation	Pass; tested capability paths and bounded output worked.

A. Interpretation

The results support three narrow conclusions. First, the tested interface routes connect Home shortcuts, recursive search and file browsing to working directory access. Second, controlled collision and trash cases demonstrate rejection of replacement and recovery in the tested scenarios. Third, terminal tests exercise preview rendering, resizing and cleanup rather than testing only isolated filesystem functions. Together these checks provide more evidence than a screenshot or successful compilation alone.

They do not show that a user completes tasks faster, makes fewer mistakes or prefers TermNav to Yazi. No latency distributions, peak memory measurements, energy figures or cross platform comparison were collected. Passing tests also cannot establish absence of data loss under every hardware failure. Claims of usability improvement and operational robustness therefore remain bounded by the implementation mechanisms and exercised scenarios.

VII. Discussion Limitations and Future Work

TermNav's main engineering value is contextual integration. Search results, previews, storage information and file operations share selection and navigation state. The command palette dispatches existing actions, making commands discoverable without introducing arbitrary shell execution. The Home view applies familiar folder and drive concepts within a terminal layout. These choices plausibly reduce context switching, but that interpretation requires a study of real users before it can be presented as a measured benefit.

A. Limits of Generalization

Platform portability remains constrained by Linux interfaces, including renameat2 and /proc descriptor paths. Windows execution relies on WSL, and a native macOS or Windows terminal port has not been established. Mount discovery is selective; recursive search deliberately avoids crossing devices. A single successful WSL verification run does not validate every Linux distribution, remote mount, permission model or terminal emulator.

Rendering has additional limits. Responsive pane changes and filename sanitization improve display stability, but automated terminal checks do not substitute for accessibility evaluation. Optional helper dependencies may be absent. Sixel negotiation and multiplexer behavior vary, and character previews retain a visible detail ceiling. Large directories can still encounter synchronous enumeration costs. Cooperative cancellation cannot interrupt every blocked read, filesystem request or process state.

The installer improves availability on supported Linux systems by building the project and arranging the executable and helpers. It is not evidence that the program installs on any computer. A bootstrap that fetches an unversioned source archive also lacks the reproducibility and verification of pinned, signed release artifacts. Public CI configuration describes intended checks; this paper reports the local run and does not claim an independently verified remote CI result.

B. Priorities for Further Evaluation

A useful next study would compare predefined navigation, discovery and recovery tasks with a shell workflow and an established terminal manager. Task completion time, error rates and user ratings should be collected with a stated participant sample and consistent datasets. Separate measurements should quantify recursive search latency, interface responsiveness during copies and peak memory on large trees. Reporting the operating system, filesystem, terminal and helper versions would make the results reproducible.

Reliability work should test interruption between metadata and payload changes, storage exhaustion, concurrent filesystem modifications and unsupported no replacement behavior. Improvements may include a documented recovery reconciliation process, standardized trash interoperability and preservation of additional file metadata. Deployment work should prioritize versioned releases, checksums and a compatibility matrix before broader platform claims. These additions should respond to observed failures or measured needs rather than expand the architecture speculatively.

VIII. Conclusion

TermNav implements a coherent terminal workspace for directory navigation, filename discovery, recoverable deletion, operation monitoring, storage inspection and content preview. Each problem is addressed by an identifiable mechanism: shared navigation state, bounded asynchronous scans, a private persistent trash registry, a worker dashboard, filesystem capacity and allocation measurements, and capability aware viewers. Fresh repository checks passed at the inspected revision and support functional behavior in their exercised scenarios. The project remains a Linux and WSL application with explicit recovery, preview and portability limits. Further usability studies and failure injection are needed to substantiate broader efficiency and reliability claims.

REFERENCES

All web references were consulted on 6 October 2026. Repository descriptions refer to the revision below; external documentation provides technical context rather than comparative experimental evidence.

[1] TermNav project repository. Commit 13599afee55c65c6a8e44523d6c1e87c3806f8f4. Primary artifact for implementation and tests.

[2] Yazi contributors. Yazi official repository and feature documentation. https://github.com/sxyazi/yazi

[3] Thomas E. Dickey and ncurses contributors. curs_threads manual. Thread support and limitations in ncurses. https://invisible-island.net/ncurses/man/curs_threads.3x.html

[4] Microsoft. Working across Windows and Linux file systems. Windows Subsystem for Linux documentation. https://learn.microsoft.com/en-us/windows/wsl/filesystems

[5] Linux man pages project. statvfs manual. Filesystem capacity and availability fields. https://man7.org/linux/man-pages/man3/statvfs.3.html

[6] Linux man pages project. rename manual. Linux renameat2 and RENAME_NOREPLACE semantics. https://man7.org/linux/man-pages/man2/rename.2.html

[7] Ramendik, M., Faure, D., Larsson, A., and Lortie, R. Desktop Trash Can Specification. Version 1.0, 2014. https://specifications.freedesktop.org/trash/latest/

Artifact and Reproduction Note

Implementation references are src/main.c, src/app.h, src/input.c, src/ui.c, src/fs_ops.c, src/async_ops.c, src/trash.c, src/search.c, src/usage.c, src/drives.c, src/history.c, src/commands.c, src/preview.c, src/rich_preview.c, src/image_terminal.c and src/sixel.c; scripts/preview_helper.py; and termnav.ps1. The Makefile identifies the test executables and integration scripts. These paths are relative to reference [1].

To reproduce the functional evaluation, obtain the stated revision, provide the documented build and optional preview dependencies, and run make check in a supported Linux environment. Fixture creation and terminal orchestration are contained in the tests directory. Results should be reported with the actual environment and exit status; the local success described here should not be substituted for another environment’s outcome.

Authorship note: [Author 2] through [Author 5] and the affiliation line are deliberate placeholders for completion before submission.